

Larder

Especificación técnica — Proyecto 1

Punto de Control 1

1 · El plan de una página

Propuesta de valor	La lista de la compra se genera sola a partir de una foto del ticket de la compra; a partir de ahí los productos se reponen o se añaden, de forma manual o automática. Sin fricción.
---------------------------	--

Cliente	Quién paga: el hogar (por hogar). Quién lo usa: los miembros del hogar.
----------------	---

Hoja de ruta

	P1 · 20.11.26	P2 · 22.1.27	P3 · 26.2.27
Qué se lanza	Larder + Backend + List, en React. Gratis.	Track (módulo a medida sobre Odoo) + pantalla en React Native. Empieza el cobro (14,99€/año).	Compare + comisión por afiliación con supermercados.
Horas	Larder — 17h Backend — 10h List — 30h	Track — 20h React Native — 10h	Compare — 13h Comisión — 10h (?)
Criterio de salida	• Items se loggean correctamente • API calls correctas • Reconocimiento de ticket • Automatización Lista ↔ Larder	• Cálculo correcto de deudas entre usuarios • Lectura correcta de items comprados • Export exitoso de app móvil	• Log correcto de tiendas • Comparación correcta de precios

Inversión

En dinero	Dominio: 10€ · Google Play: 25€ · IA / servidores: según demanda.
------------------	---

Cómo se gana dinero	Freemium: con plataformas como Stripe se implementa una suscripción para desbloquear funciones con IA (14,99€/año).
----------------------------	---

Riesgos

Riesgo

Solución

Colaboración

Pruebas exhaustivas.

Reconocimiento ticket – tienda

Tener varias opciones en cuenta.

Demasiadas funciones

Feedback objetivo.

2 · Los dos casos de uso

Caso de uso 1 — Marcar un item como "out" en Larder

Quién

Cualquier miembro del hogar.

Pasos

- 1. Ver productos con sus estados.
- 2. Click en "out".
- 3. La app guarda el cambio y lo refleja.
- 4. Se añade a la lista de la compra.

Resultado

El estado del producto queda actualizado en la base de datos y aparece en la pantalla List sin que nadie lo anote manualmente.

Qué puede fallar

- Falta de conexión.
- Dos personas cambian el mismo producto a la vez.

Caso de uso 2 — Marcar como comprado un item dentro de List

Quién

Cualquier miembro del hogar.

Pasos

- 1. Sacar foto al ticket.
- 2. El item se repone automáticamente y se borra de la lista de la compra.

Resultado

List siempre refleja el estado real de Larder.

Qué puede fallar

Falta de conexión.

3 · Delimitación con el ERP (Sistemas de Xestión Empresarial)

Esta es la sección más importante del documento — la decisión más evaluada, y la que sale directamente del estado del arte.

Con Odoo

1. Inventario — pantalla Larder

- Datos maestros: nombre, categoría, unidad, fecha de caducidad.
- Base de datos de productos existente, con alta automática.

2. Reparto de gastos — pantalla Track

- Datos maestros: hogar, miembro/usuario, precio.

Dudas: monetización con Stripe o con Odoo.

Lo corrige: Delimitación con el ERP (CE3.1) · Instalación y parametrización (CE2.2, CE2.5, CE2.7) · Datos maestros (CE2.6).

4 · La sección crítica (Servicios y Procesos)

El recurso compartido

Interactuar a la vez con el mismo ítem.

La operación en conflicto

- Larder: marcar el mismo ítem como "out" dos veces en la lista — imposible.
- List: comprar dos veces es imposible porque habría que ir a la tienda a la vez.

Qué pasa hoy

Pendiente de definir.

Qué debería pasar

Pendiente de definir.

La tarea en segundo plano

- Restock automático de Larder.
- Marcado de "comprado" automático.
- Recuento de deudas automático.
- Evento: compra automática.

Lo corrige: Concurrencia (CE2.5, CE2.6) · Tarea en segundo plano (CE1.6, CE2.2) · Depuración y documentación (CE2.8).

5 · Modelo de datos e integraciones (Acceso a Datos)

Entidades

- Hogar — id, nombre
- Usuario — id, nombre, email, hogar_id
- Producto — id (referencia al producto en Odoo), nombre, categoría, código de barras
- LarderItem — id, hogar_id, producto_id, estado (bien / queda_poco / se_acabo), actualizado_en, actualizado_por
- Listaltem — id, hogar_id, producto_id, estado (pendiente / comprado), creado_en, comprado_en, comprado_por
- LarderEvento — id, larder_item_id, estado_anterior, estado_nuevo, fecha

Relaciones

- Hogar 1—N Usuario, 1—N LarderItem, 1—N Listaltem
- Producto 1—N LarderItem, 1—N Listaltem
- Usuario 1—N LarderItem (actualizado_por), 1—N Listaltem (comprado_por)
- LarderItem 1—N LarderEvento

Claves

- Cada entidad tiene su id como clave primaria. Producto usa como identificador el id del producto en Odoos (no se duplica la clave, solo se referencia).
- LarderItem, Listaltem y LarderEvento llevan las claves foráneas correspondientes a Hogar, Producto y Usuario.

Operaciones por caso de uso

Caso 1 — Marcar producto	Leer LarderItem (mostrar lista) · Modificar LarderItem (cambiar estado) · Crear LarderEvento (registrar el cambio) · Crear Listaltem (si pasa a se_acabo).
Caso 2 — Consultar la lista	Leer Listaltem (mostrar pendientes) · Modificar Listaltem (marcar comprado) · Modificar LarderItem (volver a "bien").

Las integraciones

Desde	Hacia	Qué dato viaja	Cuándo
Interfaz web (React)	Backend	Cambio de estado de un producto en Larder	Al tocar un producto
Backend	PostgreSQL	Nuevo estado + evento de Larder	Al confirmar el cambio
Backend	Odoos	Consulta de nombre/categoría de un producto	Al mostrar un producto nuevo
Odoos	Backend	Ficha del producto (nombre, categoría)	Respuesta a la consulta
Interfaz web (React)	Backend	Marcar ítem de lista como comprado	Al tocar "comprado" en List
Backend	PostgreSQL	Actualización de Listaltem + LarderItem	Al confirmar la compra

Decisión clave: los datos de Larder y List viven en PostgreSQL, propio del backend. Odoos solo se consulta para el catálogo — nunca escribe en la base de datos de Larder ni al revés.

Lo corrige: Modelo de datos (CE2.5) · Persistencia y transacciones (CE2.4, CE2.6, CE2.10) · API y consumo (CE2.8) · Conector con el ERP (CE2.4, CE6.4).

Mapa de navegación

Dos pantallas en P1: Larder y List, accesibles desde una barra de navegación simple. Desde Larder se puede ir a List y viceversa; no hay más pantallas en esta fase.

Wireframe de la pantalla clave (Larder)

Lista vertical de productos del hogar. Cada fila muestra el nombre del producto y un indicador visual de su estado (bien / queda poco / se acabó); tocar la fila cicla al siguiente estado. Cabecera con el nombre del hogar y acceso a List.

Los tres estados de cada pantalla

Cargando	Skeleton de las filas de producto mientras se cargan los datos del hogar.
Vacía	Mensaje "Aún no hay productos en tu nevera" con una breve indicación de cómo añadir el primero.
Con error	Mensaje "No se pudo cargar" con botón "Reintentar", sin perder lo que ya estaba en pantalla si lo había.

Componentes repetidos (mínimo 3)

- Tarjeta de producto — usada en Larder y en List.
- Selector de estado (bien / queda poco / se acabó) — usado en Larder.
- Indicador de sincronización ("guardando...", "actualizado") — usado en ambas pantallas.
- Barra de navegación — usada en toda la app.

La pantalla que va a React Native

Cuál	Larder — marcar productos.
Por qué esa	Es el gesto espontáneo: se hace en la cocina, con el móvil en la mano, en el momento en que se ve que algo se ha acabado — no delante de un ordenador.
Qué cambia	Interacción más táctil (toque grande, sin hover), menos productos visibles a la vez (scroll), conectividad intermitente (la cocina puede tener mal wifi).
Qué se guarda en local	El último estado conocido de cada producto, y los cambios pendientes de enviar si no hay conexión — para que la app funcione offline y sincronice al recuperar la red.

Lo corrige: Prototipo nativo (CE2.1, CE2.2) · Ciclo de vida y persistencia (CE2.6) · Análisis de adaptación (CE1.1, CE1.2).